

ANALYSE DES BESOINS



2.1. Étude de l'existant

Avant d'entamer la phase de conception de la nouvelle application, j'ai procédé à une étude approfondie de l'existant, en collaboration étroite avec les bibliothécaires et les responsables techniques. Cette phase d'analyse s'est déroulée sur une période de deux semaines et a permis de dresser un état des lieux précis des processus métier actuels, des outils utilisés et des attentes des différents acteurs.

« Comprendre l'existant pour mieux construire le nouveau. »

Processus de gestion du catalogue

Le catalogage des nouveaux ouvrages s'effectuait dans le SIGB existant selon les normes internationales UNIMARC. Chaque notice bibliographique comprenait les métadonnées suivantes : titre, auteur(s), éditeur, année de publication, ISBN, cote de rangement, résumé, mots-clés, exemplaires disponibles. Ces informations étaient saisies manuellement par les agents du pôle acquisitions. Le système ne proposait aucune fonctionnalité de récupération automatique de notices depuis des bases bibliographiques nationales (BnF, Sudoc), obligeant à une double saisie fastidieuse.

Processus d'emprunt et de retour

Lorsqu'un étudiant souhaitait emprunter un ouvrage, il devait présenter sa carte d'étudiant à l'accueil. Le bibliothécaire recherchait manuellement le lecteur dans le SIGB en saisissant son numéro de carte, vérifiait qu'il n'avait pas atteint le quota maximal d'emprunts simultanés (fixé à 5 ouvrages), scannait le code-barres de l'ouvrage, et validait l'emprunt dans le système. Une date de retour était automatiquement calculée (14 jours pour les étudiants de licence, 30 jours pour les enseignants-chercheurs). En cas de retard, aucune alerte automatique n'était générée.

Processus de réservation

Aucun système de réservation en ligne n'était en place. Lorsqu'un ouvrage était déjà emprunté, l'étudiant devait laisser ses coordonnées sur un registre papier, et un bibliothécaire était censé le rappeler lorsque le document était à nouveau disponible. Ce processus, entièrement manuel, s'avérait peu fiable et générait des insatisfactions.

Recherche documentaire

Le catalogue en ligne, accessible via une interface web rudimentaire, permettait uniquement une recherche simple par titre, auteur ou ISBN. L'absence de recherche plein texte, de filtres avancés (par date, par catégorie, par disponibilité) et de tri des résultats rendait l'expérience utilisateur frustrante.

Gestion des pénalités

En cas de retard dans la restitution d'un ouvrage, une pénalité était appliquée selon un barème défini : 0,50 € par jour de retard pour les ouvrages courants, 1 € par jour pour les ouvrages en forte demande. Le calcul de ces pénalités s'effectuait manuellement par les bibliothécaires, sans outil d'automatisation, ce qui générait des erreurs et des contentieux fréquents avec les étudiants.



Constats de l'étude préliminaire

- ❖ Processus métier actuels trop dépendants d'interventions humaines répétitives
- ❖ Manque d'automatisation entraînant des erreurs fréquentes
- ❖ Absence d'outils en ligne allant à l'encontre des attentes des étudiants
- ❖ Impossibilité d'exploiter des statistiques d'usage pour le pilotage stratégique

2.2. Besoins fonctionnels

À partir de l'étude de l'existant et des entretiens menés avec les utilisateurs finaux (bibliothécaires, responsables administratifs, étudiants), j'ai élaboré une liste exhaustive des besoins fonctionnels que devait couvrir la nouvelle application. Ces besoins ont été hiérarchisés selon leur criticité et leur priorité de mise en œuvre.

2.2.1. Gestion du catalogue bibliographique

◆ Ajout d'un nouvel ouvrage

Saisie des métadonnées complètes : ISBN, titre, auteur(s), éditeur, année, langue, résumé, catégorie, emplacement. Upload d'image de couverture et de PDF.

◆ Modification et suppression

Modification des informations d'un ouvrage existant. Suppression après validation par un administrateur.

◆ Gestion des exemplaires

Suivi de chaque exemplaire (disponible, emprunté, en réparation, perdu). Gestion des exemplaires disponibles en temps réel.

◆ Recherche et consultation

Recherche avancée multi-critères, résultats triables et paginés. Fiche détaillée avec disponibilité en temps réel.

◆ Génération de codes QR

QR code unique par exemplaire facilitant l'inventaire et la localisation physique.

◆ Catalogue numérique

Gestion des PDF d'ouvrages numériques avec streaming sécurisé et contrôle des droits.

2.2.2. Gestion des utilisateurs

◆ Création de comptes

Trois rôles : administrateur, bibliothécaire, étudiant.

◆ Authentification sécurisée

Login par email/matricule, mot de passe sécurisé, récupération par email.

◆ Gestion des profils

Consultation et modification des informations personnelles.

◆ Historique personnel

Consultation des emprunts passés et en cours, des pénalités.

2.2.3. Système d'emprunts et de retours

◆ Création d'un emprunt

Vérification automatique des conditions (quota, retards, disponibilité). Calcul de la date de retour selon profil. Génération PDF récapitulatif.

◆ Retour d'un ouvrage

Enregistrement par scan QR. Calcul automatique des pénalités en cas de retard.

◆ Prolongations

Demande de prolongation (une seule autorisée). Validation par bibliothécaire sous réserve de non-réservation.

◆ Notifications automatiques

Emails : confirmation emprunt, rappel J-3, alerte retard J+1, récapitulatif mensuel.

2.2.4. Système de réservations

◆ Réserve en ligne

Réservation depuis le catalogue quand tous les exemplaires sont empruntés. File d'attente horodatée.

◆ Notification disponibilité

Email automatique à l'utilisateur lorsque l'ouvrage devient disponible.

2.2.5. Génération de rapports et statistiques

◆ Rapports d'activité mensuels

Emprunts, inscriptions, retards, pénalités, taux d'occupation.

◆ Statistiques d'usage

Ouvrages les plus empruntés, profils des emprunteurs actifs, taux de rotation.

◆ Inventaire annuel

Génération de listes par emplacement, vérification, identification des manquants.

◆ Export de données

Formats CSV, Excel pour traitements externes.

2.3. Besoins non fonctionnels

Au-delà des fonctionnalités métier, l'application doit répondre à des exigences non fonctionnelles garantissant sa qualité, sa pérennité et son acceptabilité par les utilisateurs.

Performance

Chargement des pages < 2 secondes. Opérations critiques < 1 seconde.

Capacité de charge

Gestion simultanée de 100 utilisateurs connectés.
Support de plusieurs dizaines de milliers d'ouvrages.

Évolutivité

Architecture permettant l'ajout de nouvelles fonctionnalités. Scalabilité horizontale.

Optimisation

Mécanismes de cache, requêtes SQL optimisées, lazy loading.

Sécurité

Protection contre injections SQL, XSS, CSRF.
Chiffrement des connexions.

Accessibilité

Interface responsive compatible mobile, respect des normes WCAG.

L'ensemble de ces besoins a servi de cahier des charges pour les phases de conception et de développement de l'application BibliothèqueUniv.

CAHIER DES CHARGES TECHNIQUE



3.1. Choix technologiques

Le choix des technologies constitue une décision stratégique majeure pour la réussite du projet. Après une analyse comparative approfondie des solutions disponibles sur le marché, j'ai retenu une stack technologique moderne, éprouvée et largement adoptée.

3.1.1. Framework backend : Laravel 12

◆ Architecture MVC claire

Séparation nette entre logique métier, présentation et contrôle du flux.

◆ ORM Eloquent puissant

Manipulation objet des données sans requêtes SQL complexes.

◆ Système de migration

Versioning de la base de données pour déploiements sécurisés.

◆ Authentification intégrée

Mécanismes d'authentification et gestion des rôles prêts à l'emploi.

3.1.2. Langage serveur : PHP 8.2

⚡ Performances

Optimisations du moteur Zend.

🎯 Typage strict

Détection d'erreurs à la compilation.

📌 Attributs

Métadonnées pour documentation.

3.1.3. SGBD : MySQL 8.0

Fiabilité ACID, performances optimisées, sécurité avancée, compatibilité native avec Laravel Eloquent.

3.1.4. Frameworks frontend

◆ Bootstrap 5

Composants pré-stylisés, système de grille responsive, développement rapide.

◆ Tailwind CSS

Framework utilitaire, flexibilité supérieure pour designs personnalisés.

3.1.5. Outils de développement

- ✦ Environnement : Docker et Docker Compose
- ✦ Contrôle de version : Git avec dépôt GitHub
- ✦ Intégration continue : GitHub Actions
- ✦ Gestion des dépendances : Composer (PHP) et npm (JS)
- ✦ Éditeur : Visual Studio Code

3.2. Architecture logicielle

3.2.1. Pattern MVC (Model-View-Controller)

L'architecture MVC est le fondement de l'organisation du code dans Laravel. Elle sépare l'application en trois couches distinctes :

Modèles

Représentation des données et logique métier.

Vues

Présentation des données à l'utilisateur (Blade).

Contrôleurs

Orchestration du flux, réception des requêtes.

Cette architecture facilite la maintenance, les tests unitaires et l'évolution indépendante de chaque couche.

CONCEPTION ET ARCHITECTURE DU SYSTÈME



4.1. Architecture globale du système

L'architecture globale de l'application BibliothèqueUniv suit un modèle three-tier (trois niveaux) classique, séparant clairement les responsabilités.



Couche Présentation

HTML5, CSS3, JavaScript.
Interface responsive, requêtes
HTTP RESTful.



Couche Métier

PHP 8.2 / Laravel 12. Logique
métier, API RESTful, ORM
Eloquent.



Couche Données

MySQL 8.0. Stockage persistant,
intégrité des données.

4.2. Diagrammes UML

4.2.1. Acteurs principaux du système



Administrateur

Gestion complète du catalogue,
des comptes, statistiques.



Bibliothécaire

Emprunts/retours, réservations,
rapports d'activité.



Étudiant

Consultation catalogue,
réservations, suivi des
emprunts.

4.3. Patterns de conception

Repository Pattern

Abstraction couche données,
facilite les tests unitaires.

Service Layer Pattern

Logique métier encapsulée
dans services réutilisables.

Observer Pattern

Système de notifications
automatiques.

4.4. Mécanismes de sécurité

4.4.1. Authentification

Authentification basée sur JWT pour les API RESTful et sessions PHP sécurisées pour l'interface web. Mots de passe hachés avec Argon2. Sessions limitées à 30 minutes d'inactivité.

4.4.2. Autorisation basée sur les rôles (RBAC)

Système RBAC garantissant que chaque utilisateur ne peut accéder qu'aux fonctionnalités autorisées pour son rôle. Protection des routes par middleware d'autorisation.

4.4.3. Protection contre les vulnérabilités web

Injections SQL

Requêtes préparées + ORM Eloquent.

Cross-Site Scripting (XSS)

Échappement des sorties HTML, CSP.

CSRF

Tokens CSRF générés par Laravel.

Clickjacking

En-tête X-Frame-Options.

Cette architecture sécurisée garantit la protection des données sensibles des utilisateurs et l'intégrité des opérations de la bibliothèque.